



INGENIERÍA MECATRÓNICA

Ciberfisica

**Actividad: U4A5. (20%) REPORTE TÉCNICO FINAL.
Demo y documentación**

Integrantes:

Samantha Elena Camacho Badillo

Angel Daniel Cisneros Perez

Ruben Alvarez Duran

Juan David Castruita Castañeda

Docente: Osbaldo Aragon Banderas

Fecha: 2026-06-10

Índice de contenido

- 1. Objetivo general del prototipo..... 1
- 2. Descripción breve del problema resuelto..... 1
- 3. Arquitectura final del sistema..... 2
- 4. Parámetros de adquisición.....3**
- 5. Dataset y preparación de datos..... 4
- 6. Descripción del modelo usado..... 6**
- 7. Funcionamiento del gemelo digital y salida generada..... 8
 - 8. Evidencias de funcionamiento..... 9
 - 8.2. Link del repositorio..... 9
 - 8.1. Link Video..... 9
- 9. Tabla resumen de resultados..... 10**
- 10. Organización del repositorio..... 10
- 11. Lecciones aprendidas..... 11
 - Qué funcionó correctamente..... 11
 - Qué no funcionó o presentó limitaciones..... 12
 - Qué se mejoraría en una siguiente versión..... 12
- 12. Conclusión técnica final..... 12
- 13. Anexos técnicos..... 13
 - 13.1 Muestra de JSON publicado por MQTT..... 13
 - 13.2 Fragmento relevante del firmware..... 13

1. Objetivo general del prototipo

Desarrollar, integrar y demostrar un gemelo digital del robot seguidor de línea como sistema ciberfísico con inteligencia artificial (CPS + IA). El prototipo adquiere telemetría real del robot físico, procesa las muestras, ejecuta un modelo de clasificación y genera una salida útil para supervisar el estado de operación del robot: OK, aviso o alerta técnica.

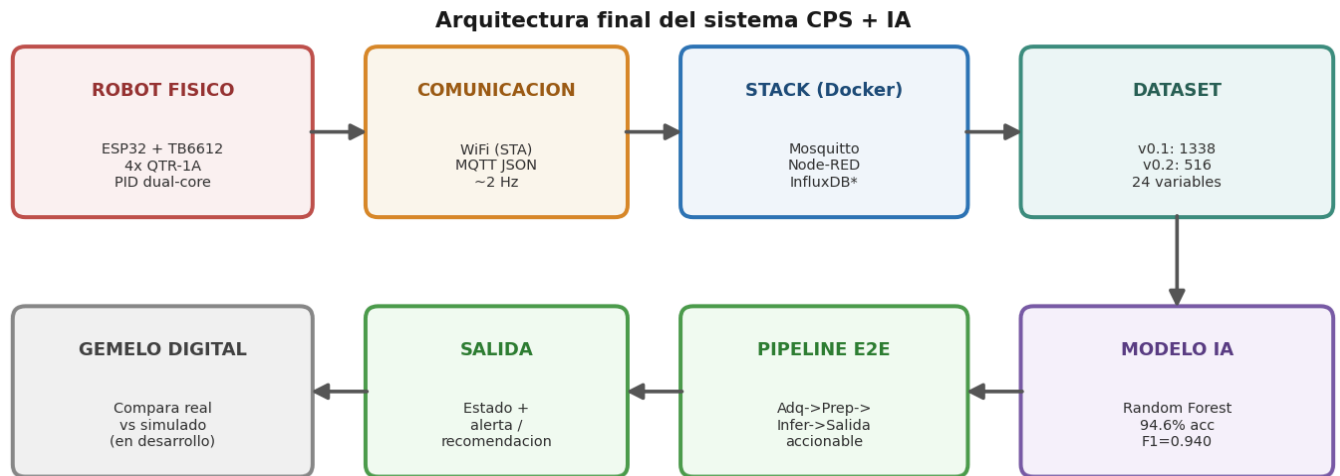
El objetivo final no es únicamente que el robot siga una línea, sino que el comportamiento físico pueda registrarse, analizarse y compararse mediante una capa digital. Con esto se obtiene evidencia objetiva para validar ajustes de control, detectar desviaciones y documentar el desempeño del sistema.

2. Descripción breve del problema resuelto

El robot seguidor de línea funciona con sensores reflectivos y control PID; sin embargo, observar físicamente no siempre permite identificar con precisión cuándo está centrado, desviado o en recuperación. Además, las pruebas repetidas consumen batería, desgastan motores y complican la trazabilidad de los resultados.

El proyecto resuelve este problema conectando el robot físico con una infraestructura de datos y un modelo de IA. La telemetría registrada permite convertir el funcionamiento del robot en un dataset, entrenar un clasificador de estados y ejecutar un pipeline de extremo a extremo que emite recomendaciones técnicas en tiempo real o en modo replay desde CSV.

3. Arquitectura final del sistema



* InfluxDB integrado en el stack (almacenamiento opcional). El flujo va del robot al dataset, y de ahí al modelo y la salida accionable.

Figura 1. Arquitectura final del sistema CPS + IA.

La arquitectura final se compone de cuatro bloques integrados:

- Robot físico: ESP32, driver TB6612FNG, motores DC y cuatro sensores reflectivos QTR-1A. Ejecuta el control PID, lee sensores y actualiza las velocidades PWM.
- Comunicación: red WiFi dedicada mediante hotspot RobotNet, IP fija del robot 192.168.137.29 y broker MQTT en 192.168.137.1:1883.
- Stack de datos: servicios Docker con Mosquitto, Node-RED e InfluxDB. Mosquitto recibe telemetría, Node-RED permite supervisión/registro y el dataset CSV conserva las muestras usadas para el modelo.
- Capa de IA/gemelo digital: pipeline Python que carga o entrena el modelo, limpia muestras, ejecuta inferencia y transforma el resultado en una salida accionable.

4. Parámetros de adquisición

Parámetro	Especificación técnica
Controlador principal	ESP32 con Arduino Core 3.x, WiFi STA, servidor web local y tarea de control en núcleo 1.
Sensores	4 sensores QTR-1A analógicos conectados a GPIO 36, 39, 34 y 35.
Actuadores	2 motores DC mediante driver TB6612FNG; pines STBY 13, AIN1 14, AIN2 27, PWMA 26, BIN1 25, BIN2 33, PWMB 32.
Periodo de control	5 ms, equivalente a una frecuencia interna aproximada de 200 Hz.
Publicación MQTT	Cada 500 ms durante diagnóstico, aproximadamente 2 Hz.
Topic principal	seguidor/telemetry; topic de diagnóstico seguidor/debug.
Variables publicadas	contador, estado, posición, error, corrección, vellzq, velDer, velBase, pérdida de línea, sensores calibrados, sensores crudos, IP, RSSI y uptime.
Formato del dataset	CSV con 24 columnas: timestamp, prueba_id, tipo_pista, contador, estado, posición/error, velocidades, sensores crudos/calibrados, RSSI, uptime y observaciones.
Condiciones de prueba	Pista de línea negra sobre superficie mate, iluminación de interior, pruebas con rectas, curvas y recuperación ante pérdida de línea.

5. Dataset y preparación de datos

El repositorio conserva dos versiones del dataset. La versión v0.1 mantiene todos los registros capturados durante la integración, mientras que la versión v0.2 elimina estados no útiles para el entrenamiento dinámico, como detenido y calibrando. Esto permite que el modelo aprenda el comportamiento del robot durante el movimiento real.

Dataset	Muestras	Variables	Uso
v0.1 crudo	1338 muestras	24 variables	Incluye detenido, calibrando, centrado, desviado_izq, desviado_der y recuperación.
v0.2 limpio	516 muestras	24 variables	Conserva centrado, desviado_izq, desviado_der y recuperación; usado para entrenamiento/evaluación.

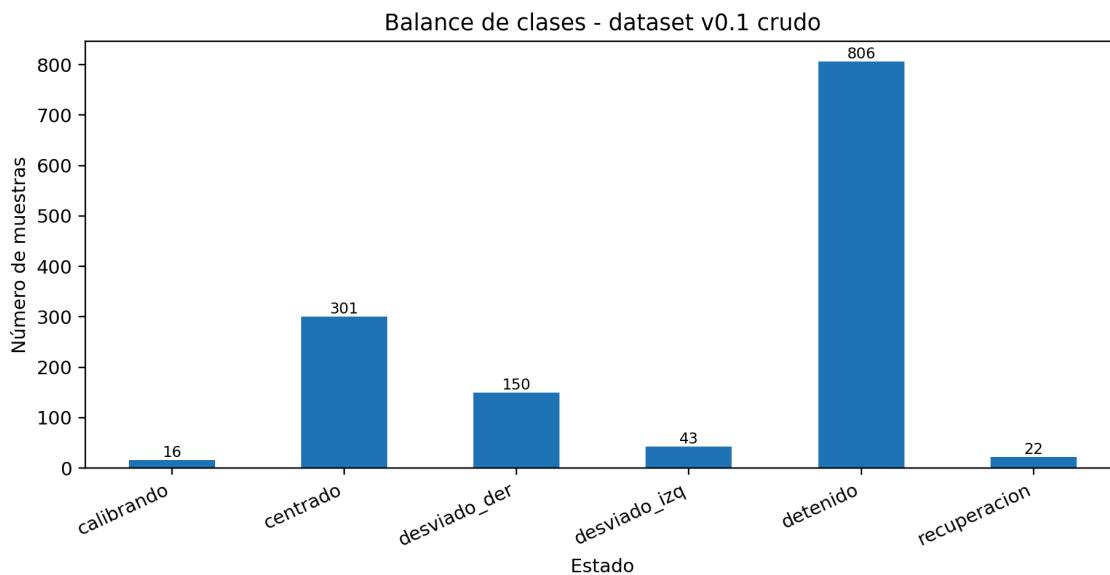


Figura 2. Balance de clases del dataset v0.1 crudo.

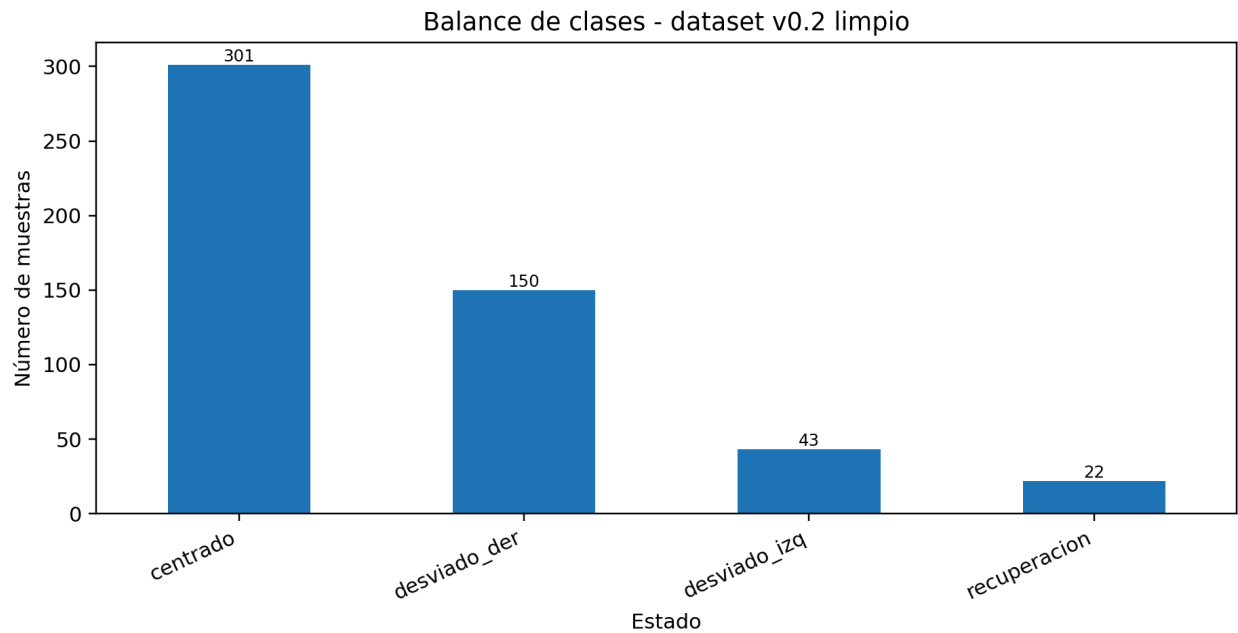


Figura 3. Balance de clases del dataset v0.2 limpio.

La limpieza del dataset fue necesaria porque el estado detenido dominaba la versión cruda. Al retirarlo, el entrenamiento se enfoca en las condiciones donde el robot realmente toma decisiones de seguimiento. Aun así, el conjunto limpio mantiene desbalance: centrado y desviado_der tienen más ejemplos que desviado_izq y recuperación. Esta limitación se reporta porque afecta la capacidad de generalización del modelo.

6. Descripción del modelo usado

Aspecto	Detalle
Tipo de modelo	Clasificación multiclase supervisada.
Modelo seleccionado	Random Forest con 100 árboles y semilla 42.
Variable objetivo	estado: centrado, desviado_izq, desviado_der, recuperación.
Variables de entrada	s0_cal, s1_cal, s2_cal y s3_cal.
Decisión técnica clave	Se evita usar error_qtr como entrada principal para reducir fuga de información, ya que ese valor ya resume la posición de línea calculada por el algoritmo QTR.
Partición	75% entrenamiento y 25% prueba, estratificado por clase.

Modelo	Accuracy	F1-macro
Baseline clase mayoritaria	0.581	0.184
Árbol de decisión	0.930	0.921
KNN (k=5)	0.946	0.932
Random Forest (seleccionado)	0.946	0.940

El Random Forest fue seleccionado porque igualó el accuracy de KNN y obtuvo mejor F1-macro. La métrica F1-macro es importante en este proyecto porque el dataset está desbalanceado; no basta con acertar la clase más frecuente.

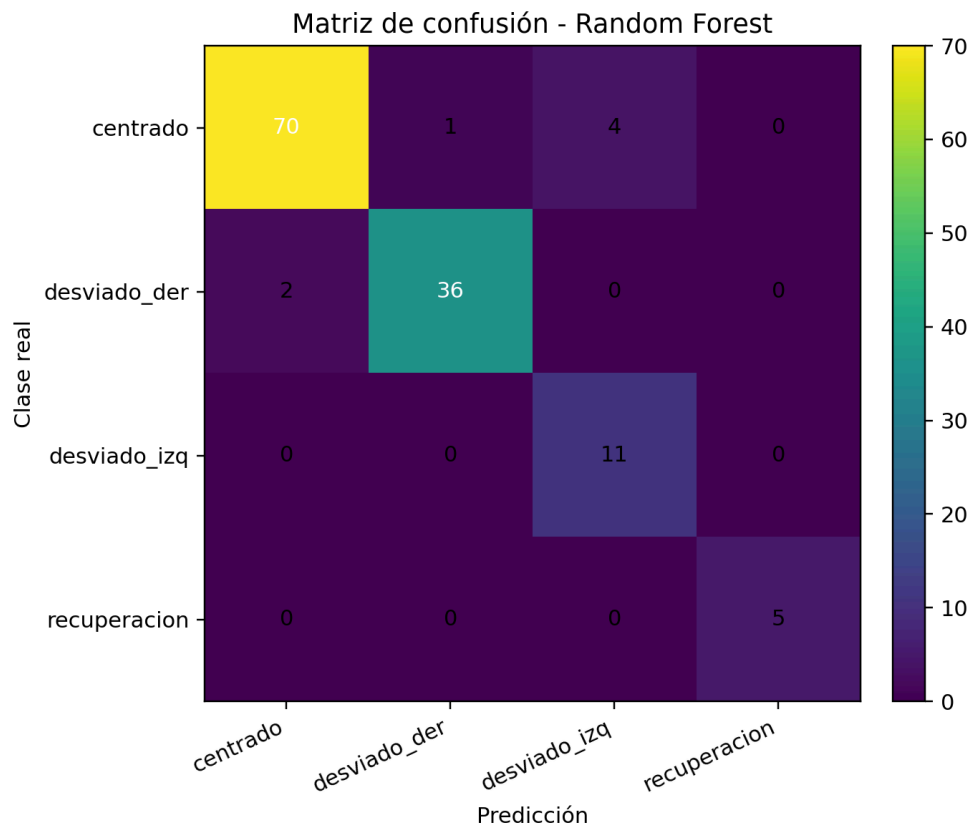


Figura 4. Matriz de confusión del modelo Random Forest en la partición de prueba.

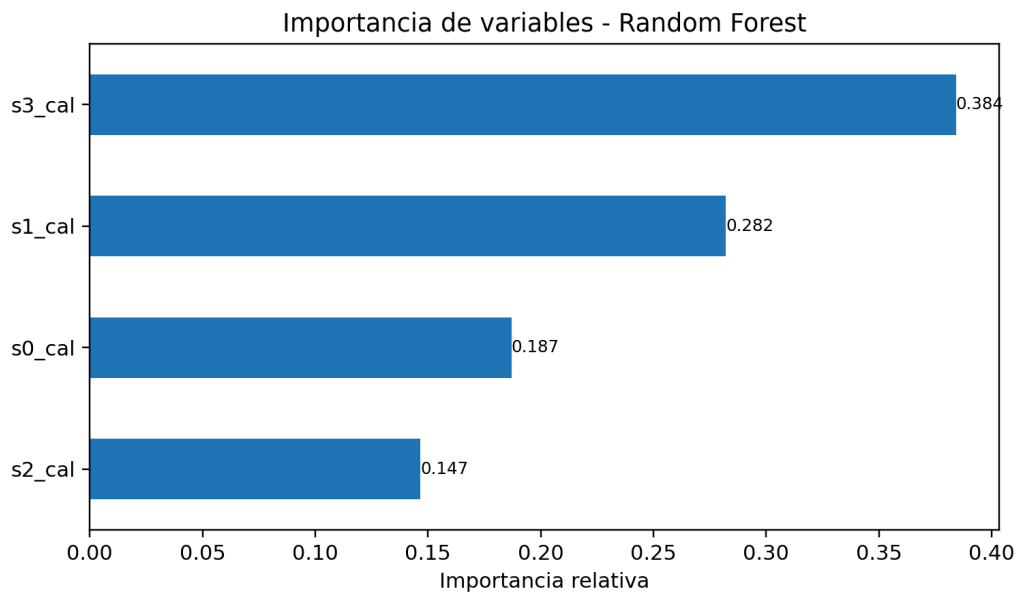


Figura 5. Importancia relativa de los sensores calibrados.

7. Funcionamiento del gemelo digital y salida generada

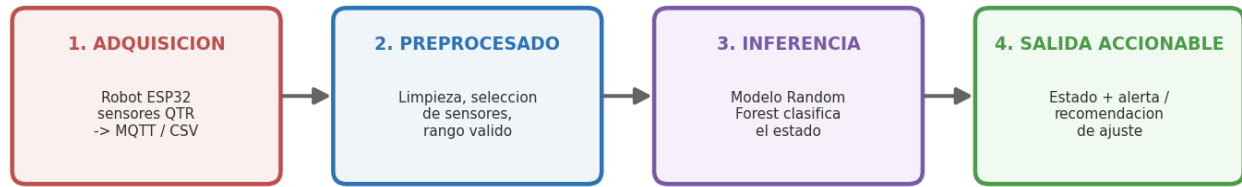


Figura 6. Flujo lógico del pipeline de extremo a extremo.

El pipeline puede operar en modo CSV, para reproducir muestras del dataset, o en modo MQTT, para recibir telemetría en vivo del robot. En ambos casos aplica el mismo procesamiento: selección de sensores, validación de valores, recorte al rango 0-1000, inferencia y generación de una salida accionable.

Estado inferido	Nivel	Salida accionable
centrado	OK	Robot centrado en la línea; operación normal.
desviado_der	AVISO	Tendencia hacia la derecha; revisar balance de motores o aumentar Kp.
desviado_izq	AVISO	Tendencia hacia la izquierda; revisar balance de motores o aumentar Kp.
recuperacion	ALERTA	Pérdida de línea; revisar trazado, velocidad en curvas cerradas o condiciones de iluminación/sensores.

8. Evidencias de funcionamiento

```
#####
# PIPELINE E2E - GEMELO DIGITAL ROBOT SEGUIDOR DE LINEA #
# Adquisicion -> Preprocesado -> Inferencia -> Salida #
#####
[MODELO] Cargando modelo entrenado desde modelo_seguidor.joblib

=====
MODULO CSV-REPLAY (Adquisicion simulada desde el dataset)
=====
[ OK ] #1 sensores=[0, 748, 756, 9] -> centrado | Robot centrado en la linea. Operacion normal.
[ AVISO] #2 sensores=[2, 13, 768, 486] -> desviado_der | Desviacion a la DERECHA. Sugerencia: el robot tiende a la derecha; revisar balance de motores o subir Kp.
[ OK ] #3 sensores=[1, 682, 789, 273] -> centrado | Robot centrado en la linea. Operacion normal.
[ OK ] #4 sensores=[1, 738, 842, 10] -> centrado | Robot centrado en la linea. Operacion normal.
[ OK ] #5 sensores=[2, 751, 842, 11] -> centrado | Robot centrado en la linea. Operacion normal.
[ AVISO] #6 sensores=[0, 5, 118, 458] -> desviado_der | Desviacion a la DERECHA. Sugerencia: el robot tiende a la derecha; revisar balance de motores o subir Kp.
[ OK ] #7 sensores=[13, 748, 779, 7] -> centrado | Robot centrado en la linea. Operacion normal.
[ OK ] #8 sensores=[1, 722, 788, 8] -> centrado | Robot centrado en la linea. Operacion normal.
[ OK ] #9 sensores=[1, 721, 807, 9] -> centrado | Robot centrado en la linea. Operacion normal.
[ AVISO] #10 sensores=[0, 65, 706, 31] -> desviado_der | Desviacion a la DERECHA. Sugerencia: el robot tiende a la derecha; revisar balance de motores o subir Kp.
[ OK ] #11 sensores=[2, 694, 747, 6] -> centrado | Robot centrado en la linea. Operacion normal.
[ OK ] #12 sensores=[7, 644, 493, 12] -> centrado | Robot centrado en la linea. Operacion normal.
[ OK ] #13 sensores=[2, 708, 884, 8] -> centrado | Robot centrado en la linea. Operacion normal.
[ AVISO] #14 sensores=[38, 209, 291, 35] -> desviado_izq | Desviacion a la IZQUIERDA. Sugerencia: el robot tiende a la izquierda; revisar balance de motores o subir Kp.
=====
RESUMEN: 14 muestras procesadas | 4 avisos/alertas generados
Conteo por estado clasificado: {'centrado': 10, 'desviado_der': 3, 'desviado_izq': 1}
=====
```

Figura 7. Ejecución real del pipeline: clasificación y salida accionable por muestra.



Figura 8. Panel de depuración (debug) de Node-RED con la telemetría JSON recibida

8.2. Link del repositorio

<https://github.com/danilopert-ui/U4A5.-REPORTE-T-CNICO-FINAL.-Demo-y-documentaci-n>

8.1. Link Video

<https://youtu.be/0jpXJQq2BpQ>

9. Tabla resumen de resultados

Prueba realizada	Condición de pista	Resultado obtenido	Métrica o indicador usado	Observación técnica
Captura inicial de telemetría	Pista de línea negra con rectas y curvas	1338 muestras en dataset v0.1	Número de muestras	Se obtuvo trazabilidad completa: timestamp, sensores, estado y variables de control.
Limpieza y filtrado de dataset	Se retiraron detenido y calibrando	516 muestras en dataset v0.2	Número de muestras / balance de clases	El dataset limpio queda enfocado en estados dinámicos para entrenamiento.
Entrenamiento de modelo	Dataset v0.2 estratificado 75/25	Random Forest seleccionado	Accuracy 0.946 / F1-macro 0.940	Supera ampliamente al baseline de clase mayoritaria.
Prueba de inferencia CSV	Replay de muestras reales	Clasificación y salida accionable funcional	Conteo de estados y avisos	Permite demostrar el gemelo digital sin depender del robot conectado.
Prueba de inferencia MQTT	Robot publicando en seguidor/telemetría	Pipeline recibe muestras en vivo	Conectividad MQTT / actualización de datos	Valida integración física-digital del sistema CPS.
Recuperación ante pérdida de línea	Curva cerrada o salida temporal de la pista	Estado recuperación/alerta técnica	Nivel ALERTA	Ayuda a identificar condiciones donde conviene bajar velocidad o ajustar PID.

10. Organización del repositorio

El repositorio entregable contiene los elementos mínimos solicitados: dataset, scripts/notebooks, modelo entrenado, evidencias, reporte final y README con instrucciones de ejecución.

```

seguidor-stack/
├── docker-compose.yml
├── mosquitto-config/mosquitto.conf
├── firmware/seguidor_linea_v3_mqtt.ino
├── dataset/
│   ├── v0.1/datos_crudos.csv
│   └── v0.2/datos_limpios.csv
├── pipeline/
│   ├── pipeline_e2e.py
│   ├── modelo_seguidor.joblib
│   └── datos_limpios_v0.2.csv
├── notebooks/U4A3_notebook.ipynb
├── documentacion/U4A5_Apellido_Nombre.pdf
├── evidencias/
│   ├── capturas/
│   └── video/enlace_video.txt
└── README.md

```

Acción	Comando o indicación
Levantar servicios	<code>docker compose up -d</code>
Ejecutar demo desde CSV	<code>cd pipeline && python pipeline_e2e.py --modo csv</code>
Ejecutar demo en vivo	<code>cd pipeline && python pipeline_e2e.py --modo mqtt --broker 192.168.137.1</code>
Revisar entrenamiento	Abrir <code>notebooks/U4A3_notebook.ipynb</code> en Jupyter o Google Colab.

11. Lecciones aprendidas

Qué funcionó correctamente

- La integración CPS de extremo a extremo: robot físico, MQTT, dataset, modelo y salida accionable quedaron conectados en una misma arquitectura.
- El control PID del ESP32 permitió obtener telemetría representativa del seguimiento de línea, incluyendo estados centrado, desviado y recuperación.
- El modelo Random Forest alcanzó buen desempeño con solo cuatro variables de entrada, lo que demuestra que los sensores calibrados contienen información suficiente para clasificar el estado operativo.

- El modo CSV del pipeline fue útil como respaldo de demostración, ya que permite validar inferencia aunque el robot o la red presenten fallas en vivo.

Qué no funcionó o presentó limitaciones

- El dataset quedó desbalanceado: centrado y desviado_der tienen más muestras que desviado_izq y recuperación.
- La comunicación MQTT depende de que el hotspot, Docker y Mosquitto estén activos; cualquier bloqueo de firewall o cambio de IP impide la captura en vivo.
- La tasa de publicación de 500 ms es adecuada para diagnóstico, pero no representa toda la dinámica interna del control PID que corre cada 5 ms.
- El gemelo digital actual clasifica estados, pero todavía no reconstruye físicamente la trayectoria completa del robot.

Qué se mejoraría en una siguiente versión

- Construir un dataset v0.3 con más recorridos, iluminación variada y mayor cantidad de muestras de recuperación/desviación izquierda.
- Integrar un sensor inercial MPU6050 para estimar giro/orientación y aproximar la trayectoria del robot en tiempo real.
- Persistir telemetría en InfluxDB y crear tableros históricos de error, velocidad y estado inferido.
- Reducir el intervalo MQTT o enviar ventanas agregadas para comparar mejor la dinámica del control físico con la inferencia digital.

12. Conclusión técnica final

El proyecto concluye con un prototipo funcional de gemelo digital para un robot seguidor de línea. El sistema integra adquisición de datos desde el ESP32, comunicación MQTT, registro en dataset, entrenamiento de un modelo de IA y ejecución de un pipeline que transforma la inferencia en una salida útil para supervisión técnica.

Con 516 muestras limpias y cuatro sensores calibrados como variables de entrada, el Random Forest alcanzó 94.6% de accuracy y 0.940 de F1-macro, superando al baseline de clase mayoritaria. Estos resultados validan que el gemelo digital puede identificar condiciones operativas relevantes, como robot centrado, desviación lateral o recuperación por pérdida de línea.

La principal aportación del proyecto es cerrar el ciclo CPS + IA: el robot físico produce datos, el entorno digital los interpreta y el resultado ayuda a tomar decisiones de ajuste o diagnóstico. Aunque el sistema aún tiene limitaciones de dataset y no reproduce completamente la trayectoria, constituye una base sólida para una siguiente versión con sensor inercial, mayor volumen de datos y visualización histórica.

13. Anexos técnicos

13.1 Muestra de JSON publicado por MQTT

```
{
  "contador": 115,
  "estado": "centrado",
  "pos": 1432,
  "error": -68,
  "correccion": -3965,
  "vellzq": -200,
  "velDer": 200,
  "velBase": 170,
  "perdida": false,
  "s": [758, 710, 623, 663],
  "crudo": [2250, 1991, 2060, 2075],
  "ip": "192.168.137.29",
  "rssi": -49,
  "uptime_ms": 59688
}
```

13.2 Fragmento relevante del firmware

```
const char* WIFI_SSID = "RobotNet";
const char* WIFI_PASS = "seguidor2026";
IPAddress LOCAL_IP(192, 168, 137, 29);
const char* MQTT_BROKER = "192.168.137.1";
const char* MQTT_TOPIC_TELEMETRIA = "seguidor/telemetry";
const char* MQTT_TOPIC_DEBUG = "seguidor/debug";
const unsigned long INTERVALO_MQTT = 500;
const uint8_t NUM_SENSORES = 4;
const uint8_t sensorPins[NUM_SENSORES] = {36, 39, 34, 35};
const int PERIODO_CONTROL_MS = 5;
const uint16_t POSICION_OBJETIVO = 1500;
  t_crudo[i] = analogRead(sensorPins[i]);
float error = (float)posicion - (float)POSICION_OBJETIVO;
const TickType_t periodo = pdMS_TO_TICKS(PERIODO_CONTROL_MS);
const float dt = PERIODO_CONTROL_MS / 1000.0;
bool configOK = WiFi.config(LOCAL_IP, GATEWAY, SUBNET, DNS1, DNS2);
```

```
Serial.print(WIFI_SSID);  
WiFi.begin(WIFI_SSID, WIFI_PASS);  
bool ok = mqtt.publish(MQTT_TOPIC_DEBUG, mensaje, true);  
Serial.print(MQTT_BROKER);  
bool ok = mqtt.publish(MQTT_TOPIC_TELEMETRIA, payload, false);  
    Serial.print(MQTT_TOPIC_TELEMETRIA);  
qtr.setSensorPins(sensorPins, NUM_SENSORES);  
mqtt.setServer(MQTT_BROKER, MQTT_PORT);  
    WiFi.begin(WIFI_SSID, WIFI_PASS);  
    if (millis() - ultimoPub >= INTERVALO_MQTT) {
```